Developer role

Saturday, 25 July 2026 11:00 pm

Key responsibilities

A Software Developer may be responsible for:

- Designing, developing, testing and maintaining software applications.
- Writing clean, reliable and reusable front-end and back-end code.
- Developing new system features based on user and organisational requirements.
- Identifying, investigating and fixing bugs, errors and technical problems.
- Connecting applications to databases, cloud services and external systems.
- Designing and working with RESTful APIs to allow different systems to exchange information.
- Creating responsive and accessible user interfaces using modern web technologies.
- Managing relational and NoSQL databases and maintaining data accuracy.
- Improving application performance, security, reliability and usability.
- Using version-control systems to track code changes and collaborate with other developers.
- Creating automated unit tests and integration tests to confirm that software works correctly.
- Supporting software deployment through Docker, CI/CD pipelines and cloud platforms.
- Monitoring deployed applications and assisting with upgrades and maintenance.
- Applying authentication, data protection and secure-development practices.
- Preparing technical documentation, procedural instructions, user guides and support materials.
- Working closely with developers, managers, researchers, users and external stakeholders.
- Explaining technical information to people with different levels of technical knowledge.
- Managing workload priorities and completing tasks with limited supervision.
- Following organisational policies, including Work Health and Safety requirements.

Required skills

Technical skills

- Strong programming and problem-solving skills.
- Understanding of software-development principles and the software-development life cycle.
- Experience with front-end web development.
- Experience with back-end programming.
- Knowledge of relational and NoSQL databases.
- Ability to design, use and troubleshoot RESTful APIs.
- Experience with Git-based version control.
- Understanding of cloud services and application deployment.
- Knowledge of Docker and CI/CD processes.
- Ability to write unit tests and integration tests.
- Understanding of authentication, authorisation and software-security practices.
- Ability to troubleshoot application, database and deployment problems.
- Ability to learn unfamiliar programming languages, frameworks and systems.

Communication and workplace skills

- Strong written and verbal communication

- Strong written and verbal communication.
- Ability to explain technical concepts in simple language.
- Ability to prepare procedural documents, user guides and technical documentation.
- Effective teamwork and collaboration.
- Strong interpersonal and stakeholder-management skills.
- Ability to build relationships with internal and external stakeholders.
- Attention to detail.
- Initiative and independent problem-solving.
- Time management and workload prioritisation.
- Ability to adapt to changing requirements.

Tools and technologies

Category	Tools and technologies	Main purpose
Front-end development	HTML5, CSS3, JavaScript	Build the structure, design and interactive behaviour of web applications
Front-end frameworks	React, Vue.js	Build reusable and responsive user-interface components
Back-end development	Node.js, Python, Java, Ruby, PHP	Develop application logic, server functions and data-processing services
Relational databases	SQL, PostgreSQL, MySQL	Store and manage structured application data
NoSQL databases	MongoDB	Store flexible or document-based data
API development	RESTful APIs, JSON	Allow applications and services to exchange information
API testing	Postman	Send API requests and inspect responses
Version control	Git, GitHub, GitLab, Bitbucket	Track changes and support team collaboration
Cloud platforms	AWS, Microsoft Azure, Google Cloud Platform	Host applications, databases and online services
Containers	Docker	Package applications and their dependencies consistently
DevOps	CI/CD pipelines, GitHub Actions, GitLab CI or Azure DevOps	Automate testing, building and deployment
Testing	pytest, Jest, Mocha or similar tools	Create unit and integration tests
Development environment	Visual Studio Code or another IDE	Write, test and debug code
Project management	Jira, Trello or Azure DevOps Boards	Manage tasks, bugs, user stories and development priorities
Documentation	Markdown, Microsoft Word, Confluence or similar tools	Prepare technical instructions, procedures and user guides
AI assistance	GitHub Copilot or another approved AI assistant	Assist with code suggestions, explanations and troubleshooting

3 weeks plan

Three-Week Software Developer Learning Plan

The goal of this plan is to gain a basic understanding of the main tools used in Software Developer roles. The focus is breadth rather than advanced knowledge.

Week 1 — Front-End and Programming Foundations

Day	Tool or topic	Learning activity	Simple output	Learning resource
Monday	HTML5 and CSS3	Learn page structure, headings, forms, links and basic responsive design	Create a simple personal webpage	MDN HTML, MDN CSS
Tuesday	JavaScript	Learn variables, functions, arrays, objects and events	Add a button that changes text on the webpage	MDN JavaScript Guide
Wednesday	React or Vue.js	Learn components, properties and basic state	Create a reusable job-card component	React Tutorial, Vue.js Tutorial
Thursday	Python or Node.js	Learn back-end basics, functions and simple server concepts	Create a basic program or web server	Python Tutorial, Node.js Introduction
Friday	Git, GitHub and VS Code	Create a repository, commit files and push changes	Upload the Week 1 project to GitHub	GitHub Skills, VS Code Getting Started

Week 1 outcome

By the end of Week 1, I should understand basic front-end development, one front-end framework, one back-end language and version control.

Week 2 — Databases, APIs and Testing

Day	Tool or topic	Learning activity	Simple output	Learning resource
Monday	SQL and relational databases	Learn SELECT, INSERT, UPDATE, DELETE and basic joins	Create and query a small users table	SQLBolt, PostgreSQL Tutorial
Tuesday	MongoDB and NoSQL	Learn collections, documents and basic CRUD operations	Create a small products collection	MongoDB Getting Started
Wednesday	RESTful APIs and JSON	Learn GET, POST, PUT and DELETE requests	Design a basic users API	MDN API Introduction
Thursday	Postman	Send API requests and inspect JSON responses	Test GET and POST requests	Postman Learning Centre
Friday	Unit and integration testing	Learn the purpose of automated testing	Write two unit tests and one integration test	pytest Tutorial, Jest Tutorial

Week 2 outcome

By the end of Week 2, I should understand relational and NoSQL databases, API communication, API testing and basic automated testing.

Week 3 — Cloud, DevOps, Security and Workplace Tools

Day	Tool or topic	Learning activity	Simple output	Learning resource
Monday	Docker	Learn images, containers and Dockerfiles	Package a simple application in a container	Docker Get Started
Tuesday	CI/CD and GitHub Actions	Learn how code can be automatically tested and deployed	Create a basic workflow that runs tests	GitHub Actions Tutorial
Wednesday	Cloud platforms	Learn the basic purpose of AWS, Azure and Google Cloud	Complete one introductory cloud module	Microsoft Azure Fundamentals, AWS Cloud Essentials
Thursday	Authentication and security	Learn login, passwords, authorisation and secure coding basics	Add a simple login check or write security notes	OWASP Top 10, MDN Authentication
Friday	Jira, documentation and Copilot	Create tasks, document the project and use AI responsibly	Prepare a Jira board, README and short user guide	Atlassian Jira Guide, GitHub Copilot Documentation

Week 3 outcome

By the end of Week 3, I should understand basic deployment, containers, CI/CD, cloud services, security, project management and technical documentation.